前后端分离环境下Spring Security权限系统构建与实现

何立富
（九江职业大学，江西 九江 332000）

摘　要：通过引入JWT认证机制，解决了前后端分离架构下Spring Security在跨域、兼容性及分布式部署中的认证和授权难题，构建了一套动态权限管理系统，实现了用户身份的精准识别与验证。在系统架构设计层面，通过自定义登录接口、缓存技术、拦截器及自定义表达式逻辑权限控制等规划，有效提升了系统的性能、安全性与灵活性。基于角色的访问控制权限（RBAC）的功能设计，借助可视化配置界面进一步增强了系统的易操作性。经测试验证，该系统具备高度的稳定性与有效性，能够精准地控制访问权限，为相关应用系统的权限管理提供了切实可靠的解决方案。

关键词：Spring Security工具；前后端分离架构；动态化权限管理；JWT标准；基于角色的访问控制权限
DOI:10.16184/j.cnki.comprg.2025.10.001

1　概述

随着信息技术的飞速发展，各类信息系统已广泛融入企业运营、公共服务、教育教学等领域的管理工作之中。在这些系统里，权限管理是关键要素，对于系统安全稳定运行有着举足轻重的意义。它能够精确界定不同用户对系统资源的访问级别和操作权限，有效防止敏感信息被泄露，避免因非法操作而引发系统故障或业务混乱[1]。Spring Security凭借强大的功能特性、高度的可定制性及与Spring生态系统的无缝集成，成为构建Web系统权限管理体系的首选。然而，随着前后端分离架构的广泛应用，Spring Security权限管理的认证方式及前后端协作授权方式都发生了变化；同时，随着业务持续发展和系统复杂度不断提升，在实现权限管理动态化方面面临着诸多挑战。基于此，研究将紧扣这些实际需求，深入剖析如何巧妙借助Spring Security这一强大工具，精心设计并搭建一套基于前后端分离架构的动态化权限管理系统，为各领域信息系统的安全与稳定运行保驾护航。

2　Spring Security

Spring Security拥有一系列核心组件，这些组件紧密协作，共同构建起强大的权限管理体系，其中，最为主要的是认证和授权过滤器。

2.1　Spring Security认证和授权流程

Spring Security执行认证与授权校验工作，主要依赖于一系列有序排列的过滤器来实现，这些过滤器构成一个过滤器链，Spring Security认证流程如图1所示。以展示的用户名和密码登录认证流程为例展开分析，为后续前后端分离架构下的登录认证提供思路铺垫。当用户输入用户名和密码并向服务器发起登录请求时，UsernamePasswordAuthenticationFilter之类的认证过滤器会拦截此请求[2]，并从请求中精准提取用户名与密码，封装成Authentication类型的对象，随后将该对象传递给ProviderManager类的认证管理器。认证管理器如同一个精密的调度中心，根据预先配置调用适配的认证提供者，以较为常见的DaoAuthenticationProvider为例，在进行认证操作时，会调用UserDetailsService对象里的loadUserByUsername方法。该方法会根据用户名从数据库中查找匹配的用户记录，将其封装成UserDetails对象并返回。DaoAuthenticationProvider会对输入的密码和UserDetails对象中的密码值进行比对（在该过程中为保障密码安全，会对密码进行加密）。若进行的比对操作成功，认证管理器就会返回一个包含认证信息的Authentication对象，将安全上下文的信息保存至SecurityContextHolder中。此时用户在当前会话中成功通过认证成为合法用户，后续的系统交互操作将基于这一认证状态逐步展开。

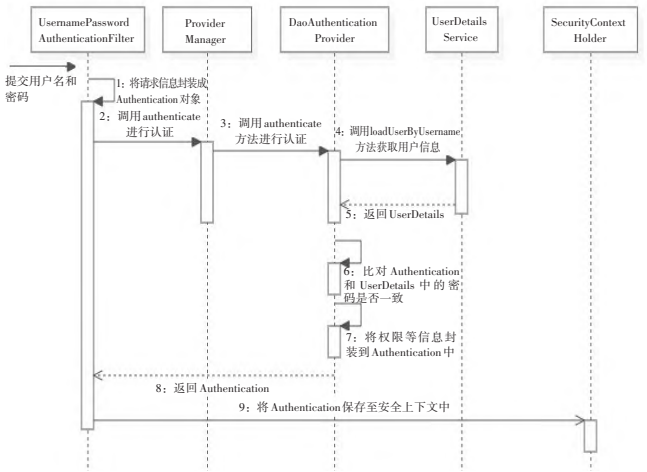

图1　Spring Security认证流程

作者简介：何立富（1979—），男，硕士，副教授，研究方向为计算机应用。

中国知网　https://www.cnki.net

用户成功认证后，若再访问受保护的特定API接口或界面资源，则Spring Security的授权流程开始发挥作用。FilterSecurityInterceptor等拦截器会即刻介入，从安全上下文中获取包含角色信息和权限详情的用户认证信息，并依据预先在配置文件中设定的或通过注解设定的授权规则进行评估。例如，基于角色的规则hasRole('ADMIN')，表明只有具备管理员角色的用户才有访问权限，权限决策管理器会整合这些信息，做出最终的权限决策。若用户权限与资源访问要求相符，则允许用户访问资源并请求放行；若用户的权限达不到要求，则抛出异常并由Spring Security按照既定配置处理，通常会向客户端返回HTTP 403 Forbidden响应，告知用户无权访问。

2.2 常规认证方式在前后端分离架构下的弊端

在前后端分离架构日益普及的当下，传统认证方式暴露出诸多问题。以Spring Security的常规认证为例，其依赖于服务器端的Session对象存储用户认证信息，客户端则借助Cookie维持与服务器Session的关联。但在前后端分离架构中，前后端常部署于不同域名或端口下，这触发了浏览器同源策略对跨域请求的限制，致使Cookie无法正常携带认证信息，导致后端服务器难以识别用户身份，从而影响了认证流程的顺利进行。同时，前后端分离架构着重强调前端的灵活性与独立性，通常会运用Vue.js、React等多种前端框架来搭建丰富多样的用户界面[3]，而部分运行这些前端应用的设备出于安全策略或自身特性等原因，并不支持存储Cookie，使传统认证方式兼容性大打折扣。此外，为提高系统的可扩展性和性能，后端服务往往采用分布式部署方式，这会产生Session共享困难等诸多问题。当用户请求在不同服务器节点间流转时，每个节点都需独立维护Session信息，这不仅提升了系统复杂度，还可能导致权限不一致问题。这些不足之处严重制约了传统认证方式在前后端分离架构中的应用。

3 系统分析与设计

系统分析与设计是整个项目的关键，其科学性与合理性直接决定系统各项功能是否能顺利实现，以及性能指标能否达标。这一环节旨在实现系统精准规划技术选型、模块划分等关键内容，从而绘制出可靠的蓝图，指引项目后续的开发与实施。

3.1 前后端分离架构下Spring Security权限管理机制分析

前后端分离架构着重于前后端解耦，赋予前端和后端各自独立开发、部署及扩展的能力。在这种架构下，后端服务应该是无状态的，即每个请求都是独立的，不依赖于之前的请求状态，这样可以提高系统的可扩展性，为实现分布式部署和负载均衡提供了便利。为解决前后端分离架构下会话管理的问题，特别是跨域请求等问题，令牌（Token）机制应运而生，例如，JSON Web Token（JWT）。通常在用户登录成功后，服务器会生成一个包含用户相关信息（例如，用户ID、用户名、权限等）的Token并返回至客户端保存。客户端在后续的请求中，将Token放在请求头或请求参数中发送给服务器，服务器通过验证Token的有效性及解析其中的用户信息来确定用户的会话状态。这一套会话技术为前后端分离架构下的交互提供了稳定且有效的方式。Spring Security底层实现为一条过滤器链，每个请求都会按顺序通过链中的过滤器，各过滤器负责执行特定的任务，并允许开发者通过添加、删除或修改过滤器来自定义安全策略。基于前后端分离架构的会话技术和Spring Security的可扩展特点，为Spring Security在前后端分离架构模式下设计一套行之有效的认证与授权实现方案奠定了基础。具体而言，可自定义登录接口，调用Spring Security中的过滤器完成认证流程；在自定义过滤器中，从请求头或请求参数中提取Token值，并根据解析出的用户信息完成权限验证流程，从而构建Spring Security在前后端分离架构下的完善且高效的认证与授权体系。

3.2 前后端分离架构下的权限管理系统设计

为了更好地满足实际应用需求，权限管理系统需具备无缝嵌入其他信息管理系统的能力，并在前后端分离架构下严格遵循高内聚、低耦合的设计原则。为给用户带来优质的使用体验，前端选用Vue.js等先进的前端框架，并通过API接口与后端进行数据交互，这不仅能够灵活适配不同信息管理系统的多样化展示需求，还能显著提升用户操作的流畅性与便捷性。在后端部分，依托Spring Boot等强大框架进行构建。无论是新搭建的系统，还是已经稳定运行的成熟系统，权限管理系统都能顺利接入，并与之实现高效协作，充分发挥权限管理的功能优势。鉴于前文对Spring Security认证和授权流程原理的详细阐述和架构需求分析，基于JWT的用户认证系统架构设计如图2所示。

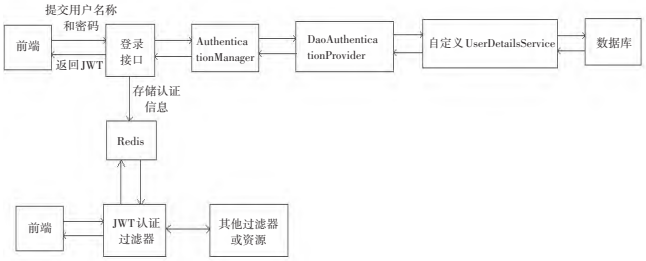

图2 基于JWT的用户认证系统架构设计

中国知网 https://www.cnki.net

3.2.1 自定义登录接口

为满足前后端分离架构下更灵活安全的认证需求，采用自定义登录接口替代Spring Security默认的基于表单登录的过滤器功能[4]，前端将用户名和密码提交至该接口后，经由AuthenticationManager、DaoAuthenticationProvider及自定义UserDetailsService进行认证。若认证成功，则系统按照令牌机制回传JWT令牌至前端存储在本地，并将认证信息以用户ID为键存储在缓存Redis中。从前端角度看，只需按登录API接口规定的规则发送登录请求即可，无需关注后端复杂的认证细节。后端则通过登录接口统一接收请求，为整个系统提供了一个集中且清晰的认证入口。这一登录接口不仅能集中处理认证逻辑，还能对前端发送的登录请求进行初步安全检查与过滤，例如，校验输入参数的合法性、防止SQL注入等攻击，并结合验证码、IP限制等安全机制，进一步增强登录环节的安全性。同时，该登录接口在系统的管理与扩展方面也颇具优势，如需增加第三方登录、社交账号登录等扩展功能，则可直接基于此接口进行开发，不会对现有系统的整体架构造成影响。

3.2.2 缓存技术

当用户完成认证后，在后续的每次请求中前端都会将令牌添加到请求头，以便后端基于此进行身份验证及权限判断，若后端频繁从数据库查询用户权限信息，则会导致数据库负载增大、响应时间变长。为解决这一问题，引入Redis作为缓存服务器，并在系统中定义Redis操作组件，实现对Redis的高效操作，从而提升系统性能。具体操作是：当后端服务器首次解析令牌并获取用户权限信息（例如，用户角色对应的权限列表）后，把这些信息存储到Redis缓存中，并设置合理的缓存过期时间。此后，若再次接收到同一用户的请求，则服务器先从缓存中查询该用户的权限信息。若缓存命中，则直接使用缓存中的权限数据进行权限验证，无需再次查询数据库，这能大幅缩短系统响应时间。当用户权限发生变动（例如，角色更新或权限重新分配）时，必须及时更新缓存中的数据，以确保系统能准确地进行权限判断。

3.2.3 JWT认证过滤器

为了达成系统安全防护、确保请求合法性这一设计目的，借助Spring Security过滤器链的可定制性这一特性，添加了JWT认证过滤器。该过滤器对所有进入系统的请求进行拦截处理，检查请求头或参数中是否携带了有效的令牌。如果没有令牌，则直接放行由后续过滤器进行权限判断。若令牌存在且有效，即表示已认证通过，就从缓存中获取用户的权限信息，并将已验证用户的认证信息存储在安全上下文中，让后续过滤器处理。这种方式使得后端无需在服务器端存储用户的会话状态，减轻了服务器的存储压力，同时便于实现分布式部署，因为每个服务实例都可以独立地对令牌进行验证，而无需依赖共享的会话存储。

3.2.4 利用自定义方法完成基于表达式的访问控制

Spring Security的授权机制灵活多样，能够满足不同业务场景下的复杂权限管理需求，但都是依据预先在配置文件中设定的或通过注解设定的授权规则来评估用户是否有权限访问相应的资源。然而当业务规则发生变化或系统功能扩展时，在配置文件或代码中预先设定授权规则的策略难以快速响应，无法根据用户的实时行为和系统的运行状态进行动态调整。为了实现更加灵活且动态的权限控制，可以利用自定义方法，依据请求的URL来判断用户是否有权限访问相应的资源。这种基于请求URL的动态权限验证机制，能够根据用户的权限及请求的具体URL进行灵活的权限判断。该方法具备更高的灵活性和可扩展性，尤其适用于需要依据业务需求动态调整权限的场景，例如，根据用户在不同业务模块中的操作权限来控制资源访问，或针对系统的不同功能模块实现权限的精细化管理。

3.3 系统功能模块设计

在信息系统权限管理范畴内，自主访问控制（DAC）、强制访问控制（MAC）及基于角色的访问控制（RBAC）是较为常用的权限管理模型[5]。DAC以用户为中心，赋予用户高度灵活性，然而在大型复杂系统中，这种灵活性易引发权限管理的混乱。MAC则基于安全标签进行访问控制，安全性极高，但其灵活性欠佳且实施成本高昂，应用场景相对受限。RBAC通过构建用户、角色、权限间的多对多关系，为不同的用户分配特定的角色，每个角色对应一组预定义的权限，与企业的组织架构适配，极大地简化了权限管理流程，是一种常见且广泛应用的方式。系统功能模块是采用基于角色访问控制的模型，系统功能模块设计如图3所示。

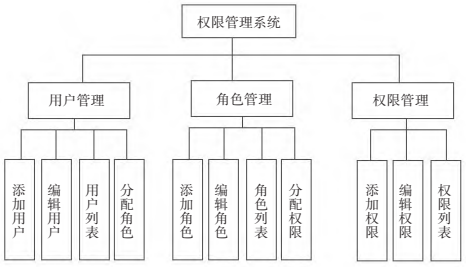

图3 系统功能模块设计

中国知网 https://www.cnki.net

在数据库设计中，围绕用户、角色和权限3个核心实体来构建系统。用户表用于存储用户基本信息，作为系统识别和管理用户的基础；角色表用于记录角色相关属性，是权限与用户间的桥梁；权限表用于定义系统中各项访问权限。用户-角色关联表通过外键关联用户表和角色表，明确用户与角色的对应关系；角色-权限关联表同理，负责建立角色与权限的联系。为了满足业务需求的动态变化，在实现方式上管理员可通过建立一个可视化的权限配置界面，直观地对用户、角色和权限进行管理。在界面上，管理员可以轻松地为用户分配或撤销权限，还可以根据业务规则设置权限的生效条件和有效期[6]。关于具体的实现方法，已有诸多文献对其进行详尽阐述，故不再做细致介绍。

4 系统核心模块实现

4.1 登录接口

按照系统架构设计，和登录接口有关的主要是自定义UserDetailsServiceImpl和LoginServiceImpl两个核心类，二者协同工作，共同构建完整的用户登录认证体系。UserDetailsServiceImpl类实现了UserDetailsService接口，在整个登录认证流程中承担着提供用户详情的重要职责，其核心方法loadUserByUsername接收用户名作为输入，根据这个用户名在数据库中找出对应的用户对象。若未找到匹配用户信息，就会抛出异常，并给出“账号或密码错误”的提示。若成功找到用户信息，就依据用户ID从数据库中查询该用户的角色列表和权限列表，并将这些信息封装成SimpleGrantedAuthority对象，添加到Authorities对象列表中。利用查询的用户信息和权限列表创建User对象并返回，为后续的认证提供必要的用户详情。LoginServiceImpl类由登录接口调用，在其中注入了AuthenticationManager对象以进行用户认证，同时注入RedisCache以实现用户信息的缓存。在login方法中，先创建一个UsernamePasswordAuthenticationToken对象，此对象会对用户的用户名和密码进行封装，之后调用AuthenticationManager来完成认证流程。若认证失败，则返回的Authentication对象为null，并抛出“登录失败”的异常提示；若认证成功，则从Authentication对象中获取用户信息并提取用户ID，并使用JWT工具类生成JWT令牌，并以用户ID值为键将用户信息缓存到Redis里。最终返回包含状态码、消息和令牌的格式化数据对象，以告知客户端登录操作已成功完成。核心代码如下。

```
public ResponseResult login(String username, String password) {
        //将用户账号和密码封装成了一个Authentication
//类型的参数
                UsernamePasswordAuthenticationToken userToken = new UsernamePasswordAuthenticationToken(username,password);
            //通过AuthenticationManager的authenticate
//方法来进行用户认证
    Authentication authenticate = authenticationManager.authenticate(userToken);
            //如果认证没通过，则给出对应的提示
    if (Objects.isNull(authenticate)) {
        throw new RuntimeException("登录失败");
    }
            //获取认证结果，并强转为LoginUser类型
    LoginUser principal = (LoginUser) authenticate.getPrincipal();
    Integer id = principal.getSys_user().getId();
            //生成jwt令牌
    String jwt = JwtUtil.createJWT(id.toString());
    Map<String, String> map = new HashMap();
    map.put("token", jwt);
    //把完整的用户信息存入redis中,userid作为key
    redisCache.setCacheObject("login:" + id, principal);
    return new ResponseResult(200, "登录成功", map);
}
```

4.2 JWT认证过滤器

在请求到达服务器时，自定义的JWT验证过滤器从请求头或请求参数中提取Token，对其进行解析和验证。如果值为空，就直接放行，由后面的过滤器去处理；否则就调用JWT组件解析出用户ID值，且根据ID值从Redis缓存中找出用户信息，并将权限信息存放到安全上下文中，以便后续的访问控制判断。在Spring Security配置类中将其配置在UsernamePasswordAuthenticationFilter之前执行，核心代码如下。

```
String token = request.getHeader("token");
  //如果没有携带token，就直接放行，由后面的过滤器
//去处理
if (!StringUtils.hasText(token)) {
    filterChain.doFilter(request, response);
    return;
 }
//解析Token，得到用户ID
String userId;
try {
    Claims claims = JwtUtil.parseJWT(token);
    userId = claims.getSubject();
} catch (Exception e) {
```


中国知网 https://www.cnki.net

```
    e.printStackTrace();
    throw new RuntimeException("token 非法");
}
//从 Redis 中获取用户信息
String redisKey = "login:" + userId;
    LoginUser loginUser = redisCache.getCacheObject
(redisKey);
if (Objects.isNull(loginUser)) {
    throw new RuntimeException("用户未登录");
}
//存入 SecurityContextHolder
        UsernamePasswordAuthenticationToken user-
namePasswordAuthenticationToken =
    new UsernamePasswordAuthenticationToken(logi-
nUser, null, loginUser.getAuthorities());
SecurityContextHolder.getContext().setAuthentication
(usernamePasswordAuthenticationToken);
filterChain.doFilter(request, response);
```

4.3 授权自定义决策器

授权自定义决策器是通过获取请求的URI与用户的认证信息进行比较来确定权限的，在该方法中需用HttpServletRequest和Authentication两个参数接收传递进来的参数值。首先，判断是否属于restful风格的地址。然后，使用该URI值和认证信息进行比较，如果存在，就返回true，否则就返回false。最后，在Spring Security配置类中使用表达式配置该自定义方法。代码如下。

```
public boolean hasPermission(HttpServletRequest re-
quest, Authentication authentication) {
  String uri="";
  try {
          //将 HTTP 请求映射到对应的控制器方法上
      requestMappingHandlerMapping. getHandler(re-
quest);
          //从 HttpServletRequest 对象里获取指定键名
//对应的属性值
    uri= (String)request. getAttribute(HandlerMapping.
BEST_MATCHING_PATTERN_ATTRIBUTE);
      } catch (Exception e) {e.printStackTrace(); }
 //判断是否是 restful 风格地址
 if(uri!=null && uri.indexOf("{")>-1){
    uri=uri.substring(0,uri.indexOf("{")-1);
 }
 if(uri!=null && uri.indexOf("?")>-1){
    uri=uri.substring(0,uri.indexOf("?")-1);
 }
//取出之前的用户认证信息
 Object obj=authentication.getPrincipal();
 if(obj instanceof UserDetails){
    UserDetails userDetails=(UserDetails)obj;
    Collection<? extends GrantedAuthority> authori-
ties=userDetails.getAuthorities();
      //如果权限信息中包含地址值，就返回 true
    return authorities.contains(new SimpleGrantedAu-
thority(uri));
  }
  return false;
}
```

5 结语

综上所述，针对前后端分离架构下Spring Security权限管理面临的挑战，构建了动态化权限管理系统。引入JWT认证机制，解决传统Session－Cookie认证在跨域、兼容性及分布式部署方面的难题，确保用户身份准确识别与验证。和当前多数权限管理系统相比，搭建的系统有着独特的创新点。一方面，系统在授权机制上，通过自定义方法实现基于表达式的访问控制，突破了传统在配置文件或代码中预先设定授权规则的局限性，能够根据用户实时行为和系统运行状态动态调整权限，显著提升了权限控制的灵活性与可扩展性。另一方面，在架构设计上综合运用自定义登录接口、缓存技术和JWT认证过滤器等，实现了高效的认证与授权流程，有效解决了前后端分离架构下的跨域、兼容性及分布式部署难题，在性能与安全性方面具有明显优势。经测试，系统稳定有效，能精确控制访问权限，为相关权限管理提供可靠方案，有望在多领域信息系统管理方面发挥重要作用。

参考文献

[1] 苏兵. 基于Spring Security的权限管理系统的设计与实现[J]. 信息与电脑, 2023(9)：24-31.
[2] 覃龙伟, 兰全祥. Spring Security中设计模式的运用浅析[J]. 大众科技, 2023(9)：1-5.
[3] 王勇，吕明久，陈莉，等. 基于RBAC模型的权限管理系统的设计与实现[J]. 江苏工程职业技术学院学报, 2024(9)：15-20
[4] 杜成龙, 曹海平, 吴亮. 基于Spring Security的自定义用户认证和授权策略研究[J]. 现代职业教育, 2023(3)：75-78.
[5] 陈海锋, 丘美玲. 基于RBAC模型的前后端分离系统设计与实现[J]. 科技创新与应用, 2024(4)：102-105.
[6] 付越, 张传国. 按钮级权限管理与统一认证系统设计与实现[J]. 电脑知识与技术, 2024(4)：44-46.

中国知网 https://www.cnki.net